

Evaluating the Generalizability of LTS: Rare-Class Text Triage Across Domains

PRAVEEN BANDLA, New York University, USA

REBECCA GLICK, New York University, USA

KRISTINA FUJIMOTO, New York University, USA

1 Introduction

Many real-world text classification tasks share a common structure: a large unlabeled corpus in which the *relevant* class is rare. This broader *data triage* problem involves efficiently discovering and labeling rare but important instances in an imbalanced corpus, and is both practically important and methodologically underexplored.

The original application of **LTS** (Learn-to-Sample) addresses this challenge in the context of wildlife trafficking detection, where the goal is to identify specific animal product listings (like sharks and leather, for example) among thousands of ordinary marketplace advertisements. LTS was proposed as a cost-effective pipeline that combines large language models with active data selection to minimize manual labeling effort while scaling detection across large corpora [1].

1.1 Problem Formulation

An open question is whether the LTS pipeline generalizes beyond its original domain. The original system was designed and evaluated on a narrow binary classification task with specific structural assumptions, including e-commerce style text, moderate class imbalance, and high-quality LLM-generated labels. It remains unclear whether these assumptions hold across substantially different datasets and classification tasks.

To evaluate whether LTS functions as a broadly applicable data triage framework rather than a domain-specific solution, we test it on datasets that vary across number of classes and degree of class imbalance.

1.2 Prior Work

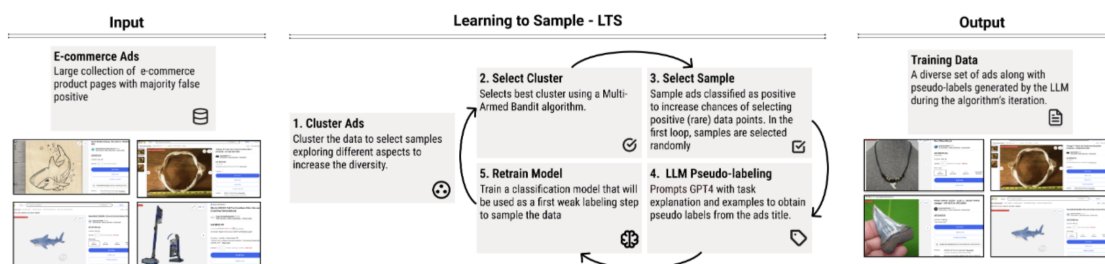


Fig. 1. Figure from prior work showing LTS pipeline.

Code available at: <https://github.com/praveen-bandla/LTS>.

Authors' Contact Information: Praveen Bandla, New York University, New York, NY, USA, pb3060@nyu.edu; Rebecca Glick, New York University, NY, USA, rm9724@nyu.edu; Kristina Fujimoto, New York University, NY, USA, kef7529@nyu.edu.

Barbosa et al. [1] addresses the critical bottleneck that happens with computationally expensive labeling. The LTS method is able to generate quality labeled data for classifiers at scale that is comparable to GPT-4 and other trillion parameter sized models. Their core insight is that leveraging in-context learning with few-shot prompting techniques on a subset of the data allows these LLM-generated labels to serve as pseudo-ground truths for training these specialized classifiers. Their experimental evaluation demonstrates that the resulting specialized classifiers for their wildlife dataset achieve up to 95% F1 score which outperforms direct LLM-based classification at a significantly lower cost. The pipeline is explained below.

1.2.1 The Original LTS Pipeline. The original LTS pipeline shown in Figure 1.2 operates as follows:

- (1) Load and preprocess an unlabeled corpus, optionally balancing the dataset via undersampling and partitioning the data into semantic clusters (e.g., using LDA topic modeling) to provide coverage across different regions of the corpus.
- (2) Select samples from clusters using a specified sampling strategy, either Thompson sampling or random sampling, where each cluster is treated as a bandit arm and prioritized based on previously observed utility.
- (3) Label selected samples using either LLM-generated annotations or pre-existing file-based labels, converting outputs into discrete class labels suitable for supervised learning.
- (4) Fine-tune a transformer-based classifier (e.g., BERT) on the accumulated labeled dataset, evaluating model performance after each iteration using a chosen metric such as F1, accuracy, precision, or recall.
- (5) Update the sampling policy based on observed improvements in downstream model performance, repeating the iterative sampling and training process until the labeling budget or stopping criterion is reached.

1.2.2 Limitations: Reproducibility and Lack of Generalizability. The original LTS paper states: “we employ a grid search to find the optimal hyperparameters for the model to improve prediction accuracy,” specifically over the learning rate and weight decay. However, inspection of the accompanying GitHub repository reveals that these values are effectively hardcoded to a learning rate of $2e-5$ and a weight decay of None, the latter of which prevents reproducibility of code (failure during execution). While we were unable to reproduce the exact results reported in the paper due to limitations in documentation, we implemented our own grid search procedure and applied it to both the original LTS pipeline and our extended data adapter framework in order to systematically evaluate the impact of hyperparameter tuning on performance.

2 Contributions

Our work makes two primary contributions:

- (1) **Reusable infrastructure for dataset generalization.** We design and implement a data-adapter framework that enables LTS to be extended to new datasets without modifying the core pipeline. This infrastructure includes per-dataset YAML configuration files and dataset-specific Python adapter subclasses. We additionally extend the original binary classification only implementation to support multiclass classification tasks as well.
- (2) **Empirical Evaluation of LTS Generalizability.** We evaluate the framework’s robustness by applying it to two diverse multiclass datasets: **Emotion Tweets** (6-class) and **Reuters-21578** (9-class) [2, 3]. These datasets introduce significant variation in type of task, number of classes, as well as class distribution in comparison to the original wildlife trafficking domain.

3 Experiment Design

3.1 Datasets

Beyond the original wildlife trafficking dataset used in the reproduced LTS pipeline, our primary evaluation datasets are Emotion Tweets and Reuters-21578. Emotion Tweets consists of 6-class social media text with labels including sadness, fear, joy, and love, while Reuters-21578 contains highly imbalanced newswire articles spanning topics such as earnings, acquisitions, and trade. All the while, both datasets preserve the strong class imbalance characteristics central to the original data triage setting, allowing us to evaluate whether LTS generalizes beyond binary rare-event detection.

Reuters-21578 is commonly transformed into the smaller Reuters R8 benchmark dataset, which restricts the corpus to the eight most frequent topic categories (earn, acq, money-fx, grain, crude, trade, interest, and wheat) while preprocessing the dataset by cleaning and removing outliers. This reduces the dataset to 7,674 documents by discarding all remaining categories.

Dataset	Task	Pool	Val	Classes
Emotion Tweets	Multiclass	16,000	2,000	6 emotions
Reuters-21578	Multiclass	9,603	3,299	9 topics

Table 1. Multiclass Extension: Emotion Tweets & Reuters-21578: Breakdown of multiclass datasets including task, pool, validation set size, and number of classes.

Rather than removing the less frequent classes entirely, we adapted the original Reuters-21578 dataset by grouping the remaining 82 categories into a single “other” class. This produced a 9-class classification task containing all 12,902 documents. We specifically chose this approach for two reasons. First, “other” remains a meaningful class label that captures more difficult to class labels including several minority classes, which is highly relevant to the Data Triage task that LTS looks to tackle. Second, we wanted to preserve the original class distribution in order to reflect more-closely a dataset we would see in the real-world as opposed to a benchmark friendly dataset.

3.2 Infrastructure Improvements

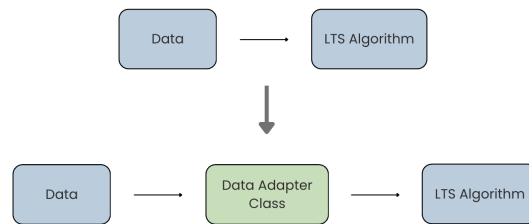


Fig. 2. The data adapter layer decouples dataset-specific logic from the core LTS pipeline. Each dataset is represented by a YAML config and a lightweight adapter subclass; the pipeline itself is unchanged.

3.2.1 Data Adapter. In the original LTS implementation, dataset-specific logic - column names, file paths, and label parsing - was hardcoded across the core pipeline modules. Adding a new dataset required editing algorithm files directly.

We restructured this by introducing a `DataAdapter` abstract base class that sits between each dataset and the pipeline. Each dataset, once preprocessed, receives a YAML configuration file and a minimal Python subclass; the core sampling and training code is left untouched. The adapter is responsible for four concerns:

- (1) **Schema mapping:** Column names are declared in the YAML and exposed through a consistent interface, so the pipeline never references dataset-specific column strings directly.
- (2) **I/O and artifact management:** The adapter resolves data file paths and manages a namespaced artifact directory for each run, including saved models, logs, results, and bandit state.
- (3) **Task semantics:** The adapter formats prompts and parses LLM responses into class labels. Binary datasets specify match strings in the YAML; multiclass subclasses override the parsing method with a `label_map` that maps LLM output strings to integer class indices.
- (4) **Trainer configuration:** The adapter constructs the BERT fine-tuner with dataset-specific settings, including the number of output labels and model variant.

An `AdapterRegistry` maps a dataset key (e.g., `emotions`, `reuters`) to the corresponding adapter class at runtime, so the pipeline entry point requires only a single command-line argument to select the correct dataset configuration.

Critically, this design means the core algorithm has no direct knowledge of any dataset. All dataset-specific information flows through the adapter interface, making it straightforward to extend LTS to a new domain without touching the sampling, labeling, or training logic.

Adding a new dataset requires only a YAML configuration file, a prompt template, a lightweight preprocessing script, and a small adapter subclass - no changes to the core pipeline. More information of how we implemented the data adapter class is located in the repository’s README under “adapters” <https://github.com/praveen-bandla/LTS>.

3.2.2 Multiclass Extension. The original LTS system supported only binary classification. We extend it to N -class problems by: (a) parameterizing `BertForSequenceClassification` with a configurable `num_labels` field read from the YAML; (b) replacing the binary-specific class balancing logic with a general N -class version using grouped sampling; and (c) overriding `parse_model_output` in multiclass adapters to map a richer label vocabulary to integer class indices.

3.2.3 Adaptive Grid Search. To optimize classification performance, the training process was transitioned to an adaptive Multi-Armed Bandit (MAB) hyperparameter search. This approach allowed the model to navigate a predefined grid of hyperparameters—including learning rates, weight decay, and data subsets (bandits)—in a sequential manner. Rather than treating each trial as an independent experiment, the architecture utilized sequential weight inheritance, where each new iteration initialized its weights from the previously saved state. This created a continuous “learning chain” that refined the model’s feature extraction while the Bandit algorithm balanced exploration of new parameters, like weight decay coefficients, with the exploitation of settings that yielded the highest F1 scores.

During implementation, we encountered significant hardware constraints related to GPU memory (VRAM) overhead when using standard BERT batch sizes. To resolve these Out-of-Memory (OOM) errors, we optimize the training pipeline by reducing the per-device batch size and implementing gradient accumulation. By decoupling the physical batch size from the optimization step, we maintain a stable effective batch size while staying within the hardware’s memory limits, ensuring the sequential bandit iterations complete without further interruption.

3.3 Methodology

All experiments use 10 active learning rounds, Thompson sampling with $K = 5$ LDA clusters, and weighted F1 as the primary evaluation metric. We inference GPT-4o-mini for labelling, called using OpenAI API. The classifier is bert-base-uncased fine-tuned as described in Section 3. More details on experimental constants can be seen in Table 2. All experiments were conducted using the NYU HSRN Kubernetes cluster infrastructure.

Category	Parameter	Value
Model	Fine-tune Model	bert-base-uncased
	Dropout	0.2
LTS Pipeline	Sampling Type	Thompson
	Clusters (k)	5
	Sample Size (n)	400
	Labeling Oracle	gpt-4o-mini
Optimization	Iterations	10
	Max Epochs	20
	Metric to Optimize	F1
	Early Stopping (Patience)	5
	Temperature (τ)	0.2
	Class Balancing	False

Table 2. Experimental Hyperparameters and Configuration Constants

4 Results

4.1 Wildlife Dataset

Wildlife		
Metric	Benchmark (Original LTS)	Final Model
F1	0.87	0.94
Eval Accuracy	0.95	0.94

Table 3. LTS performance on the Wildlife dataset (Shark Dataset) - benchmark retrieved from the original paper

Table 3 compares the performance of our reproduced LTS pipeline against the benchmark results reported in the original work. Using the same underlying LTS framework while incorporating our adaptive grid search procedure, the final model achieved an F1 score of 0.94 compared to the reported benchmark F1 score of 0.87, suggesting that the adaptive grid search yielded stronger model performance on the baseline. Evaluation accuracy remained comparable to the original work, decreasing only slightly from 0.95 to 0.94.

Table 4 provides more detail into how the model evolved throughout the iterations using the adaptive grid search method. Early rounds consistently selected Bandit 4, with model performance steadily improving from an initial F1 score of 0.519 to a plateau in the low 0.83–0.84 range. During the final iteration, we note the largest performance jump when the sampling policy selected Bandit 0, resulting in a final F1 score of 0.936 and evaluation accuracy of 93.79%.

This suggests that the fixed budget of 10 active learning iterations may have constrained the exploration process. Additional rounds may have allowed the Thompson sampling policy to further explore informative regions of the dataset and potentially improve classifier performance beyond the final reported score.

Round	Bandit	Accuracy	F1	Eval Loss
1 (Iteration 0)	Bandit 4	46.89%	0.519	0.906
2 (Iteration 1)	Bandit 4	72.14%	0.696	0.914
3 (Iteration 2)	Bandit 4	77.35%	0.754	1.006
4 (Iteration 3)	Bandit 4	76.95%	0.761	0.728
5 (Iteration 4)	Bandit 4	83.37%	0.832	1.019
6 (Iteration 5)	Bandit 4	83.37%	0.845	0.493
7 (Iteration 6)	Bandit 4	83.77%	0.836	0.672
8 (Iteration 7)	Bandit 4	83.17%	0.832	0.569
9 (Iteration 8)	Bandit 4	82.57%	0.830	0.641
10 (Iteration 9)	Bandit 0	93.79%	0.936	0.725

Table 4. Wildlife metrics reported per round: bandit, accuracy, F1, evaluation loss.

Overall, these results suggest that the original LTS framework remains reproducible even after introducing substantial infrastructure modifications. At the same time, because the LTS pipeline contains stochastic components through Thompson sampling within clusters, repeated runs of the same task may produce slightly different performance trajectories and final results.

4.2 Multiclass Datasets

Dataset	Metric	Final Model
Emotions	F1	0.62
	Eval Accuracy	0.62
Reuters	F1	0.82
	Eval Accuracy	0.81

Table 5. LTS performance on Emotion Tweets (6-class) and Reuters-21578 (9-class)

Looking at Table 5, we observe that despite both datasets introducing substantially greater class complexity than the original wildlife trafficking task, Reuters-21578 achieved comparatively strong performance. This may suggest that the clearer topical separation present in newswire data allows the LTS sampling and clustering pipeline to more effectively identify informative regions of the dataset. Additionally, the Reuters dataset preserves strong class imbalance characteristics that closely align with the original data triage motivation behind LTS, making it a more structurally similar extension of the original task despite the increased number of classes.

In contrast, Emotion Tweets presented a substantially more difficult classification setting. Unlike topic-oriented datasets, emotion classification depends heavily on contextual tone and semantic nuance, where categories such as sadness, fear, and love often overlap in vocabulary and meaning. This reduced the effectiveness of LDA-based clustering, which relies primarily on bag-of-words topic separation. Additionally, the dataset itself contains a degree of inherent labeling noise and ambiguity, further complicating both pseudo-label generation and downstream classifier training.

Across both datasets, experiments were conducted under the same fixed budget of 10 active learning iterations and 5 LDA clusters. As the multiclass search space expanded, these constraints likely limited the ability of the Thompson sampling policy to sufficiently explore informative regions of the dataset before convergence. Nevertheless, these results establish an initial baseline demonstrating that the LTS framework can extend beyond binary rare-event detection into multiclass classification settings.

5 Discussion

5.1 Generalization of the LTS Framework

Our results demonstrate that the core LTS framework can be generalized beyond its original wildlife trafficking domain with relatively minimal changes to the underlying pipeline. The introduction of the data adapter layer enabled new datasets to be integrated through standardized YAML configuration files, dataset-specific preprocessing scripts, and modular label mapping logic without requiring modifications to the core sampling or training workflow. This significantly reduced the engineering overhead associated with evaluating LTS on new tasks and improved the overall reproducibility of the system compared to the original implementation.

Additionally, extending the original binary-only implementation to support multiclass classification demonstrated that the broader LTS framework is not fundamentally restricted to rare-event binary detection tasks. The wildlife dataset results further suggest that Thompson-sampling-based selection remains effective when semantic clusters are relatively well-separated and labels are coherent. In these settings, the adaptive sampling strategy was able to rapidly prioritize informative regions of the dataset and improve downstream classifier performance across successive rounds of fine-tuning.

5.2 Empirical Findings

Performance varied substantially across datasets, suggesting that the effectiveness of LTS depends heavily on dataset structure and class complexity. Reuters achieved relatively strong performance despite the multiclass setting, likely because newswire topics contain more distinct vocabulary and clearer topical separation. In contrast, Emotion Tweets produced substantially lower performance, suggesting that LTS struggles more when class boundaries are semantically overlapping or context-dependent.

For Reuters-21578, our modified 9-class formulation preserved the full dataset and its real-world imbalance characteristics instead of restricting evaluation to the cleaner R8 benchmark. However, grouping 82 categories into a single “other” class likely introduced substantial semantic heterogeneity that weakened downstream classification performance. Despite these constraints, the results still establish a useful baseline demonstrating that LTS can extend beyond binary classification settings into larger multiclass domains.

The Emotion Tweets dataset introduced additional challenges due to the semantic overlap between emotion categories such as sadness, fear, and love. Unlike topic-oriented datasets, emotion classification depends more heavily on contextual tone and semantic nuance, limiting the effectiveness of bag-of-words-based LDA clustering. These findings suggest that LTS may generalize more effectively to datasets with clearer topical separation than to highly nuanced semantic classification tasks.

Across both multiclass datasets, the fixed experimental budget of 10 active learning rounds likely constrained the ability of Thompson sampling to sufficiently explore informative clusters before convergence. This limitation becomes increasingly important as the number of classes and overall search space expand.

5.3 Limitations

Several limitations of this work should be considered when interpreting the results. First, the fixed experimental configuration of 10 rounds and 5 LDA clusters constrained exploration in the multiclass settings, particularly for Reuters and Emotion Tweets where the search space was substantially larger than the original wildlife trafficking task.

Additional rounds may have allowed the sampling policy to better converge toward informative regions of the data distribution.

Second, our evaluation did not include direct comparisons against alternative labeling and training strategies such as full-corpus LLM labeling or standard supervised BERT fine-tuning. As a result, it remains difficult to fully quantify the cost-performance tradeoff introduced by the LTS framework relative to simpler baselines.

Finally, the use of LDA clustering introduces representational limitations, as bag-of-words topic modeling may fail to capture the semantic nuance required for emotionally ambiguous or highly heterogeneous classes. This likely reduced the effectiveness of cluster-guided sampling on datasets where semantic meaning extends beyond surface-level vocabulary patterns.

5.4 Future Work

Future work should evaluate the latency, computational cost, and labeling cost of LTS relative to alternative approaches such as full-corpus LLM labeling and standard supervised BERT training. In particular, comparing LTS against a baseline untrained BERT classifier trained directly on labeled data, as well as pipelines relying entirely on full-corpus LLM annotation, would provide a clearer understanding of the practical tradeoffs between accuracy, runtime, and labeling efficiency. Additionally, benchmarking LTS against other active learning and sampling strategies would help determine whether the adaptive Thompson-sampling framework provides measurable advantages over simpler or more computationally efficient alternatives.

These comparisons would help future users of LTS better evaluate whether the additional complexity introduced by active sampling and iterative retraining is justified under real-world resource constraints. Establishing standardized benchmarks across latency, API cost, computational overhead, and downstream classification performance would also provide a stronger foundation for future optimization work.

Future experiments should additionally explore larger experimental budgets, including increased active learning rounds and alternative cluster counts, to better understand how exploration depth affects convergence and downstream performance in multiclass settings. Finally, several components of the pipeline could be further optimized to reduce overall runtime. For example, API-based labeling calls could be parallelized across batches of samples, and portions of the iterative training and evaluation workflow could be streamlined to improve scalability for larger datasets and more computationally intensive experiments.

6 Conclusion

We presented an empirical generalizability study of the LTS (Learn-to-Sample) pipeline originally proposed for wildlife trafficking detection [1]. Our work evaluated whether the core components of LTS — LDA clustering, Thompson-sampling-based selection, LLM few-shot labeling, and iterative BERT fine-tuning—transfer effectively to new multiclassification tasks with different class-balance characteristics. Unlike the original binary wildlife trafficking task, our evaluation datasets introduced multiclass classification settings, allowing us to assess whether LTS generalizes beyond rare-event binary detection. Our work makes three primary contributions. First, we evaluated LTS on two new multiclass datasets, Emotion Tweets and Reuters-21578, using Thompson-sampling-based selection. Second, we developed a reusable data-adaptor infrastructure consisting of YAML-driven configuration files, standardized preprocessing scripts, and a unified Python adaptor interface, enabling reproducible extension of LTS to new datasets with minimal engineering effort. Third, we extended the original pipeline with systematic hyperparameter optimization through grid search, improving reproducibility and enabling more consistent performance tuning across datasets.

References

- [1] Juliana Barbosa, Ulhas Gohdhali, Gohar Petrossian, Kinshuk Sharma, Sunandan Chakraborty, Jennifer Jacquet, and Juliana Freire. A cost-effective LLM-based approach to identify wildlife trafficking in online marketplaces. *arXiv preprint*, 2024.
- [2] Praveen Govi. Emotions dataset for nlp. Kaggle, 2020. URL <https://www.kaggle.com/datasets/praveengovi/emotions-dataset-for-nlp>. Accessed: [May 2026].
- [3] The Devastator. Uncovering financial insights with the reuters-21578. Kaggle, 2022. URL <https://www.kaggle.com/datasets/thedevastator/uncovering-financial-insights-with-the-reuters-2>. Accessed: [May 2026].